# BlackJack Program

*Software Requirements Specifications*

Daniel Alvarado, Monisha Turan, Nikki Ong, Sadhvik Chelmatikary,
Tochi Ezeife

# Revision History

| Date | Revision | Description | Author |
|---|---|---|---|
| 02/20/2026 | 1.0 | Initial Version | All Members |
| 03/01/2026 | 1.1 | Added Definitions, Overview, Constraints | Nikki Ong |
| 03/02/2026 | 1.2 | Added Functional and Modules requirements | Monisha Turan |
| 03/02/2026 | 1.3 | Added Assumptions, Scope, Product Perspective & Architecture | Tochi Ezeife |
| 03/02/2026 | 2.0 | Added External & Internal Requirements | Sadhvik Chelmatikary |
| 03/02/2026 | 2.1 | Added Non-Functional Requirements | Daniel Alvarado |
| 03/04/2026 | 2.2 | Edited Format/Layout & Add Class Diagram | Nikki Ong |
| 03/04/2026 | 2.3 | Added Use Case Specs. + Sequence Diagram | Nikki Ong |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

# Table of Contents

**1. Purpose**

a. Scope 4
b. Definitions, Acronyms, Abbreviations 4
c. References 4
d. Overview 5

**2. Overall Description**

a. Product Perspective 6
b. Product Architecture 6
c. Product Functionality/Features 6
d. Constraints 7
e. Assumptions and Dependencies 7-8

**3. Specific Requirements**

a. Functional Requirements 9
    i. Common Requirements 9
    ii. User Module Requirements 10
    iii. Server Module Requirements 10
    iv. Game Logic Module Requirements 11
    v. Networking Module Requirements 11
    vi. Client/GUI Module Requirements 11-12
b. External Interface Requirements 12
c. Internal Interface Requirements 12

**4. Non-Functional Requirements**

a. Security and Privacy Requirements 13
b. Environmental Requirements 13
c. Performance Requirements 13

# 1. Purpose

This document outlines the requirements for a Multiplayer Black Jack Gaming System.

## 1. Scope

This document will detail the user, GUI, functional and non-functional requirements for the BlackJack Simulation Program (Name pending). This document will not document how these requirements will be implemented. It is a blueprint that will be closely adhered to during the design phase.

## 2. Definitions, Acronyms, Abbreviations

**Dealer** - Authorized personnel responsible for shuffling and dealing the deck of cards in accordance with Blackjack regulations

**Hand** - The cards held by the player/dealer during the game.

**Hit** - the action of acquiring a card to be dealt by the dealer to the player's hand.

**Stand** - the action of refusing to be dealt more cards from the dealer.

**Bust** - when the accumulation of cards the dealer deals exceeds 21.

**Ace** - in Blackjack, an Ace card can be represented as a value of 1 or 11

**Natural** - Player's first two cards are an Ace and a ten point card (Jack, Queen, King, or a 10 card) equaling an immediate value of 21.

## 3. References

Use Case Specifications - Page 14-20
UML Use Case Diagram - Page

Class Diagram - Page

Sequence Diagrams - Page

## 4.Overview

The Multiplayer Black Jack Gaming System, is designed to simulate a real-time casino experience, allowing multiple players to simultaneously compete against a real-time dealer. Where players aim to reach a hand value of 21 without going over that number.

# 2. Overall Description

## 1. Product Perspective

Create a multi-player blackjack gaming system. The system allows clients to log in, add funds to their accounts, play games of blackjack with other real players, or cash out their accounts. Standard rules apply. Cheating is minimized. Play is fair. This is a Java application with a GUI that operates over TCP/IP. This system requires a server application and client application. There is no web or HTML component. No databases, libraries, frameworks, or other technologies may be used without approval.

## 2. Product Architecture

The system will be organized into 5 major modules:

1. User module
2. Server module
3. Game Logic module
4. Networking module
5. Client/GUI module

Note: System architecture should follow standard OO design practices.

## 3. Product Functionality/Features

The high-level features of the system are as follows (see section 3 of this document for more detailed requirements that address these features):

- Being able to create an account and login.
- To add/withdraw funds from player's accounts.
- Allow multiple players to join and play at tables.
- Automatic table management
- Dealer Functionality, enforcing Blackjack rules.
- GUI displaying gameplay.
- Server managing game logic, player sessions, and fairness within the game.
- Communication between client application and server using network sockets.

## 4. Constraints

**2.4.1** The system must be implemented entirely in Java.

**2.4.2** Since the system has to operate as a Java application it means there mustn't have web-based code be used (HTML, CSS, JavaScript).

**2.4.3** The system must use Java socket connections due to it being a client-server application.

**2.4.4** The server has to handle client disconnections without stopping active games/other connections in the server system.

**2.4.5** The server must support multiple client connections since multiple players can play at the same time.

**2.4.6** The system has to correctly handle Blackjack standard rules such as an Ace valuation (it being value 1 or 11) & recognize a natural BlackJack on the first deal.

**2.4.7** Players must not lose money unfairly, meaning their account balance can’t drop below zero where bets must be validated by the server before each round.

**2.4.8** Only registered users with valid credentials can connect to the server, since players have to be authenticated before joining a game.

**2.4.9** The system must not allow players to access other player’s hands (each player’s cards in game) or manipulate game outcomes as the system has to be fair.

**2.4.10** The system must maintain valid decks for each table.

## 5. Assumptions and Dependencies

**2.5.1** It is assumed that the maximum # of players at a table active at any time will be 35

**2.5.2** It is assumed that the system will **not** be running at all times.

**2.5.3** It is assumed that any number of players can login at any time.

**2.5.4** It is assumed that all users have access to the internet.

**2.5.5** It is assumed that one dealer will be assigned to an active table.

# 3. Specific Requirements

## 3.1 Functional Requirements

### 3.1.1 Common Requirements:

**3.1.1.1** The system shall allow registered users to log in using a valid username and password.

**3.1.1.2** The system shall allow multiple players to connect and play simultaneously.

**3.1.1.3** Each table shall contain one Dealer and up to six Player users.

**3.1.1.4** Users shall be required to place a bet before cards are dealt.

**3.1.1.5** The system shall automatically calculate hand totals.

**3.1.1.6** The system shall determine win, loss, push, or blackjack outcomes at the end of each round.

**3.1.1.7** The system shall display real-time game updates to all players at the table.

**3.1.1.8** The system shall prevent player account balances from dropping below zero.

**3.1.1.9** A new table shall be created when all seats at the current table are occupied.

**3.1.1.10** Players that do not perform an action within a certain amount of seconds will automatically fold

**3.1.1.11** Players shall have the ability to replay and join the next round after a game concludes.

### 3.1.2. User Module Requirements:

**3.1.2.1** allows users to create an account with a unique username and password (6–20 characters).

**3.1.2.2** allows users to log in using their registered credentials.

**3.1.2.3** allows users to deposit money into their account before joining a game.

**3.1.2.4** The system shall allow users to cash out their available balance.

**3.1.2.5** displays the user's current balance on the screen.

**3.1.2.6** prevents users from placing bets higher than their available balance.

**3.1.2.7** The system shall allow users to join an available table.

**3.1.2.8** The system shall allow users to choose to hit, stand, or fold during their turn.

### 3.1.3. Server Module Requirements:

**3.1.3.1** Handle multiple players connected at the same time.

**3.1.3.2** Verify the login information before allowing access.

**3.1.3.3** Server shall shuffle and deal all cards.

**3.1.3.4** Control turn order in clockwise direction.

**3.1.3.5** Calculate hands total and determine winners.

**3.1.3.6** Update player balances after each round.

**3.1.3.7** Continue running even if a player disconnects.

### 3.1.4. Game Logic Module Requirements:

**3.1.4.1** Follow standard Blackjack rules during the game.

**3.1.4.2** Deals two cards to each player and the dealer at the start of a round.

**3.1.4.3** The system shall determine when a player busts (goes over 21).

**3.1.4.4** The dealer shall stand when their hand reaches 17 or higher.

**3.1.4.5** The system shall treat an Ace as either 1 or 11.

**3.1.4.6** Reset the game state after each round ends.

### 3.1.5. Networking Module Requirements:

**3.1.5.1** The client shall connect to the server using Java TCP/IP sockets.

**3.1.5.2** The system shall send player actions from the client to the server.

**3.1.5.3** The server shall send updated game information back to all players.

**3.1.5.4** Keeps all players' game view synchronized.

### 3.1.6. Client/GUI Module Requirements:

**3.1.6.1** Provides interactive controls for player actions (Hit, Stand, Fold)

**3.1.6.2** Sends player actions to the server

**3.1.6.3** Displays the player’s current hand of cards

**3.1.6.4** Displays the dealer’s one card faced up and other card faced down

**3.1.6.5** Displays the player’s account balance and current bet

**3.1.6.6** Displays game status messages

**3.1.6.7** Displays the results of rounds (Win, Lost, Bust)

**3.1.6.8** Client/GUI updates based on server responses

## 3.2. External Interface Requirements

**3.2.1** The system must provide an interface to the User module so that the users can see their dealt cards and make decisions in the game. The interface should be of a visual format with the following fields: cards, current balance

**3.2.2** The system must provide an interface to the User module so they can login to the system. The interface should consist of 2 text fields: username and password. The user must be able to provide input in the form of text. The system must also give a visual cue if there was an error in any of the entries. The system should suggest the user to try inputting their information again.

**3.2.3** The system must provide an interface to end their game session through a visual cue.

**3.2.4** The system must provide an interface across different computers to share their game data and progress over TCP protocol.

## 3.3 Internal Interface Requirements

**3.3.1** The system must represent each card with a different number value. The system must use the card's value to compute score
**3.3.2** The system must randomize the card of decks with built in pseudorandom generator before shuffling each time
**3.3.3** The system should fill the table with different players based on the queue of waiting clients in a first come first serve fashion.
**3.3.4** The system must be able to restrict the number of cards to be drawn after the amount 21 has been exceeded by the users

# 4. Non-Functional Requirements

## 1. Security and Privacy Requirements X

**4.1.1** User logins are password protected, users can only access an account with a valid username & password matching our records.

**4.1.2** Passwords can consist of the following: (char: A-Z, int: 0-9, symbols:_!?@#&) and are case sensitive.

**4.1.3** Username & Password details will be saved via an ordered list.

## 2. Environmental Requirements

**4.2.1** system must be able to support a Java Runtime Environment.
**4.2.2** system must have wifi capabilities and be able to connect to a network.
**4.2.3** Databases aren't used to store information.

## 3. Performance Requirements

**4.3.1** system will process players' action inputs within a second.
**4.3.2** system shall support multiple concurrent players, dealers and multiple active game tables.
**4.3.3** system will be able to support multiple game tables at a time.

Use Case Specifications:

**Use Case ID**: 001

**Use Case Name:** Player Joins Available Table

**Relevant Requirements:** * Specific Requirements: 3.1.7

**Primary Actor**: Player - Non-dealer user that will participate in game

**Pre-conditions:**

- Player is logged in.
- The dealer is present at the game table.

**Post-conditions:** Player is able to join table and dealer begins round.

**Basic Flow or Main Scenario:**

1. Player clicks "Play"
2. System checks for available game tables
3. Game table was found with an unoccupied seat.
4. Player joins table
5. The dealer begins the round.

**Extensions or Alternate Flows:**

1. A table was found, but was full (6 players occupied the 6 spots), so the system generates a new table.
2. No game tables were found, so the system generates a new table and assigns a dealer to the table.

**Exceptions**:

- The server could fail to find a table, the system displays an error in the server.

**Related Use Cases:** 012, 009, 010

—----------

**Use Case ID**: 002

**Use Case Name:** Player Action: "Hit"

**Relevant Requirements:** Game Logic: Requirements 3.4.1, 3.4.3

**Primary Actor**:

Player -Non-dealer user participating in game with the actions: hit, stand, forfeit, Dealer - User in charge of dealing cards to players, cannot hit when card total >= 17

**Pre-conditions:**

- Player must be at an available table with an active game.
- Cards must have been dealt to all players at table including dealer.
- Bets have already been placed.
- Turn order matches the current player.

**Post-conditions:** The player is given a random card from the deck.

**Basic Flow or Main Scenario:**

1. During player's turn user selects action "hit"
2. The dealer gives them a random card from the deck.
3. If hand total <= 21, user can choose to "hit" or "stand"
4. Turn order progresses to the next player.

**Extensions or Alternate Flows:**

1. When card is dealt, user hand total > 21, automatic "bust" prompt from system and bet for current round is forfeit.

**Exceptions**:

- Server error communicating request action, display an error message with the server.

**Related Use Cases:** 002, 004, 005

—----------

**Use Case ID**: 003

**Use Case Name:** Player Action: "Stand"

**Relevant Requirements:** * Game Logic: Requirements 3.4.1, 3.4.3

**Primary Actor**: Player

**Pre-conditions:**

- Player must be at an available table with an active game.
- Cards must have been dealt to all players at table including dealer.
- Bets have already been placed.
- Turn order matches the current player.

**Post-conditions:** Player turn ends.

**Basic Flow or Main Scenario:**

1. Player clicks "Stand"
2. Turn ends and moves to the next player in the round.

**Extensions or Alternate Flows:** none

**Exceptions**:

- Server error communicating request action, display an error message with the server.

**Related Use Cases:** 010

—----------

**Use Case ID**: 005

**Use Case Name:** Deposit Funds

**Relevant Requirements:** * User Module: Requirements 3.2.1- 3.2.5, Client/GUI Requirements: 3.5.2, 3.5.5, 3.5.8

**Primary Actor**: Player

**Pre-conditions:**

- Player must be logged in with their registered credentials

**Post-conditions:**

- Player's account balance is updated

**Basic Flow or Main Scenario:**

1. The player chooses the "Deposit Funds" option.
2. The system prompts the player: "Enter Amount:" to deposit.
3. The player enters the amount they want to deposit.
4. Client sends the deposit request to the server.
5. The server updates the player's account balance.
6. The system displays the updated balance to the player.

**Extensions or Alternate Flows:**

1. The player chooses to cancel the action of depositing.

**Exceptions**:

- If an invalid amount was entered (such as inputting letters), the system displays error in amount.
- Server could fail to update account balance, the system displays an error in the server.

**Related Use Cases:**

- 008

—----------

**Use Case ID**: 006

**Use Case Name:** Cash Out Funds

**Relevant Requirements:** * User Module: Requirements 3.2.1- 3.2.5, Client/GUI Requirements: 3.5.2, 3.5.5, 3.5.8

**Primary Actor**: Player

**Pre-conditions:**

- The player must be logged in with their registered credentials.
- The player is currently not playing a game.

**Post-conditions:**

- The player's account balance is withdrawn.
- Player account balance is updated.

**Basic Flow or Main Scenario:**

1. The player chooses the “Cash Out” option.
2. The system displays the player’s current balance.
3. The player confirms to proceed with withdrawal.
4. The server processes the withdrawal request.
5. The system displays “Withdrawal processed” and updates the account’s balance.

**Extensions or Alternate Flows:**

1. The player chooses to cancel the withdrawal request.

**Exceptions**:

- The server could fail processing the withdrawal request, the system displays an error in the server.

**Related Use Cases:**

- 006, 008

—----------

**UC ID**: **007**

**Use Case Name**: Create Account

**Relevant Requirements**: User Module Requirements, Security Requirements

**Primary Actor:** Player, Dealer

**Pre-Conditions:** User is not currently logged In.

**Post-Conditions**: A new user account is created and stored with username, password, and starting funds.

**Basic Flow/ Main Scenario**:

1. User selects Create Account from the main menu.
2. System prompts user to enter a username and password.
3. User enter the required information in required format.
4. System checks username is not already in use.
5. System creates the account and confirm success.

**Extensions/Alternate Flows**:

1. Username already exists - displays “Username already exists” and ask user to try again.
2. Password is Invalid - displays password rules and ask user to try again.

**Exceptions:**

- System fails to save account details, displays error message and returns to main page.

**Related Use Cases**: none

—----------

**Use Case ID**: **008**
**Use Case Name**: Start Round (Dealer Begins Game)
**Relevant Requirements**: Common Requirements , Game Logic Requirements
**Primary Actor**: Dealer
**Pre-conditions**: Dealer is at table. At least one player is seated. All seated players have placed bets.
**Post-conditions**: Round begins and initial cards are dealt.
**Basic Flow or Main Scenario**:

1. Dealer selects Start Round.
2. System checks that at least 1 dealer and 1 player are present.
3. verifies that all players have placed a bet.
4. Server starts the round and triggers initial dealing.
5. displays updated game to all users.

**Extensions or Alternate Flows:**

1. Missing dealer or player system displays “Waiting for players/dealer.”
2. One or more players haven’t bet system displays “Waiting for bets.”

**Exceptions:** Server fails to initialize round, system displays error and returns table to waiting state.
**Related Use Cases**: 005, 002

—----------

**Use Case ID:** **009**

**Use Case Name:** Deal Initial Cards

**Relevant Requirements:** Game Logic Requirements, Server Requirements

**Primary Actor:** Dealer

**Pre-conditions:** Round has started. Deck is available. Bets are locked in.

**Post-conditions:** Each player has initial cards and dealer has initial cards.

**Basic Flow or Main Scenario:**

1. Server shuffles the deck.
2. Server deals two cards to each player at the table.

3. Server deals two cards to the dealer (at least one visible card).
4. System updates all clients with starting hands (players only see their own hand + dealer visible card).
5. System sets turn order to the first player clockwise from the dealer.

**Extensions or Alternate Flows:** If deck is low on cards system resets deck and shuffles before dealing.

**Exceptions:** Network message fails to deliver updates, displays error.

**Related Use Cases:** 002, 003, 004

---

**Use Case ID: 010**

**Use Case Name:** End Round / Determine Winners / Payout
**Relevant Requirements:** Game Logic Requirements (outcomes), Server Requirements (update balances)
**Primary Actor:** Server, Dealer participates
**Pre-conditions:** All players have completed their turns OR have busted/folded. Dealer has completed dealer turn.
**Post-conditions:** Winners are determined and player balances are updated.

**Basic Flow or Main Scenario:**

1. Server confirms all players are done acting.
2. Dealer completes dealer turn (draw until 17+).
3. Server compares each player's final hand to the dealer's hand.
4. Server determines outcome (win/lose/push/blackjack) for each player.
5. Server updates each player's balance based on the result.
6. System displays results to all players and prompts replay/next round.

**Extensions or Alternate Flows:** Player folded- player automatically loses bet for that round.

**Exceptions:** Server fails during payout update, system displays error and prevents starting new round until resolved.

**Related Use Cases:** 002, 006, 007

---

**Use Case ID**: 012

**Use Case Name:** Ace Value

**Relevant Requirements:** * Game Logic Module: 3.1.4.5

**Primary Actor**: Table

**Pre-conditions:**

- The player is in a current game and it is their turn.
- The player receives an Ace.

**Post-conditions:**

- Ace value is determined to be 11 or 1 (best outcome for player).

**Basic Flow or Main Scenario:**

1. Ace card is dealt to player.
2. Server receives that Ace was drawn.
3. Server sends to Table to determine Ace value by the player's current hand total.
4. Table sends best outcome for player to Server.
5. Server sends updated Ace card + best hand total outcome to Client.
6. Client displays Ace card + hand total.

**Extensions or Alternate Flows:** none

**Exceptions**:

- Server could fail to update hand total or sending communication, the system displays an error in the server.

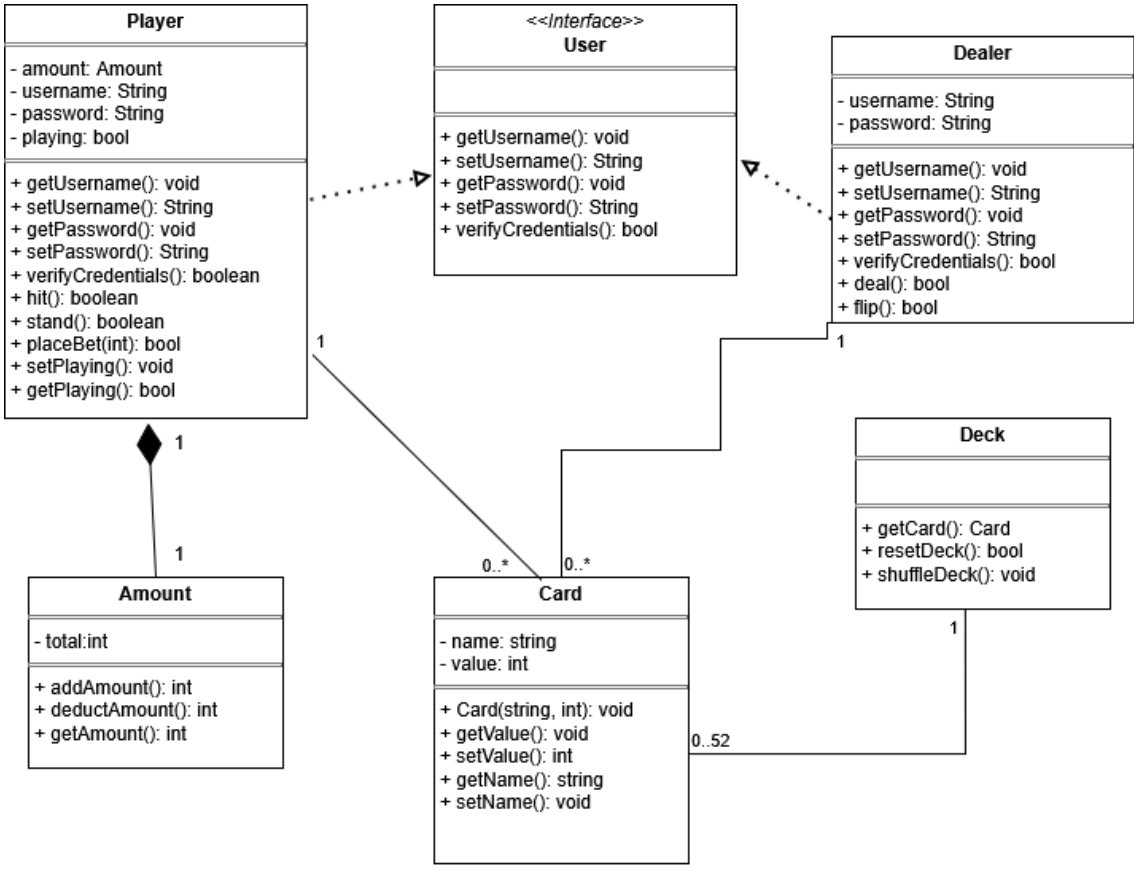
**Related Use Cases:**

- 003, 009, 010

UML Use Case Diagram:

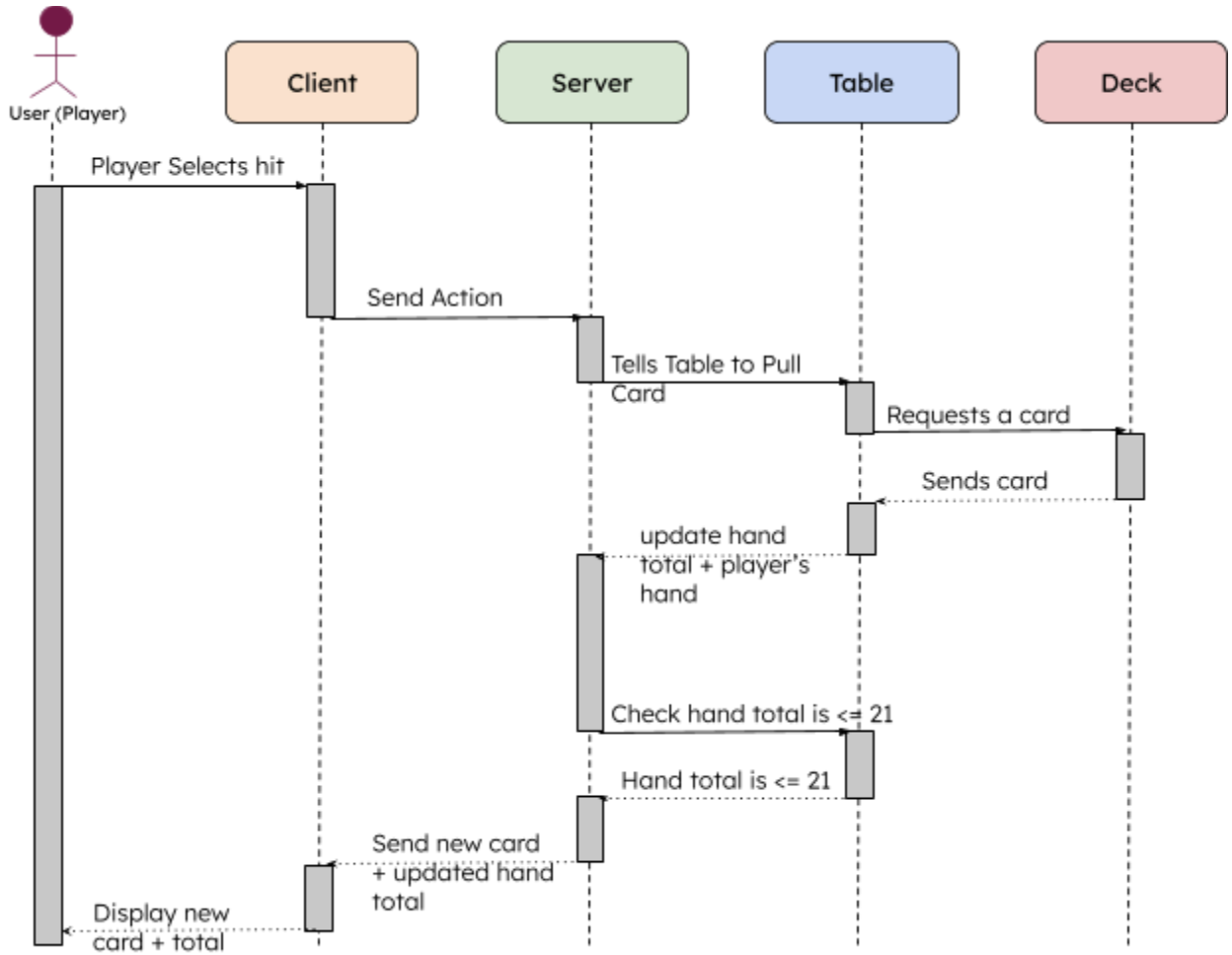
(insert use case UML here)

Class Diagram:




Sequence Diagrams:

- Player Action: “Hit”


User (Player)
Client
Server
Table
Deck
Player Selects hit
Send Action
Tells Table to Pull Card
Requests a card
Sends card
update hand total + player's hand
Check hand total is <= 21
Hand total is <= 21
Send new card + updated hand total
Display new card + total